1.Care este diferența între un microprocesor și un microcontroler?

Microprocesorul se poate defini ca fiind un circuit logic programabil de către utilizator, într-o singură capsulă şi cu funcţie de procesor de uz general; Microcontrolerul se poate defini ca un circuit logic, programabil de către utilizator, cu o structură adaptată pentru rezolvarea unei largi game de aplicaţii de timp real; el nu este folosit ca procesor de uz general ci ca procesor orientat pe aplicaţii, fiind, în general, inclus în echipamentul pe care îl conduce;

2. Care este diferența între registrele interne ale unui microprocesor și memoria operativă?

Registrii sunt locatii de stocare foarte rapide aflate in interiorul procesorului de capacitate mica cu acces direct la procesor.

Memoria operativa(Ram) este mai lenta decat registrele se afla in afara procesorului (comunica prin bus cu acesta , are capaciate mai mare de (cativa MB).

3. Descrieți execuția de către microprocesor a unei instrucțiuni cu operanzi și rezultat.

RS:

1.IF (Instruction Fetch) folosind valoarea din PC luam instructiunea si o salvam in RI (registru de instructiune). PC apoi este incrementat pentru a indica urmatoarea instructiune.

2.ID (Instruction Decode) aici se decodifica instructiunea e analizeaza codul operatiei si locatiile operanzilor , register interne , adrese de memorie sau valor immediate.

3.EX (Execute) aici se genereaza semnale pentru a se executa instructiunea pe baza la codul operatiei aceasta etapa depinde de tipul instructiunii.

4.MEM (Memory Access) Daca se specifica pe baza instructiunilor un acces la memorie pentru salvarea rezultatului sau pentru accesul unei informatii din memorie aceasta are loc la acest pas

5.WB (Write Back) Rezultatul este scris in registrul specificat in instructiune la acest pas.

4. Ieșirea unei porți logice normale (cu 2 stări) poate fi legată la o

magistrală? Dacă nu, de ce?

Pentru a putea fi conectat la magistrala avem cateva conditii

1.Uniformitate functionala - rolul si scopul liniilor sa fie aceleasi

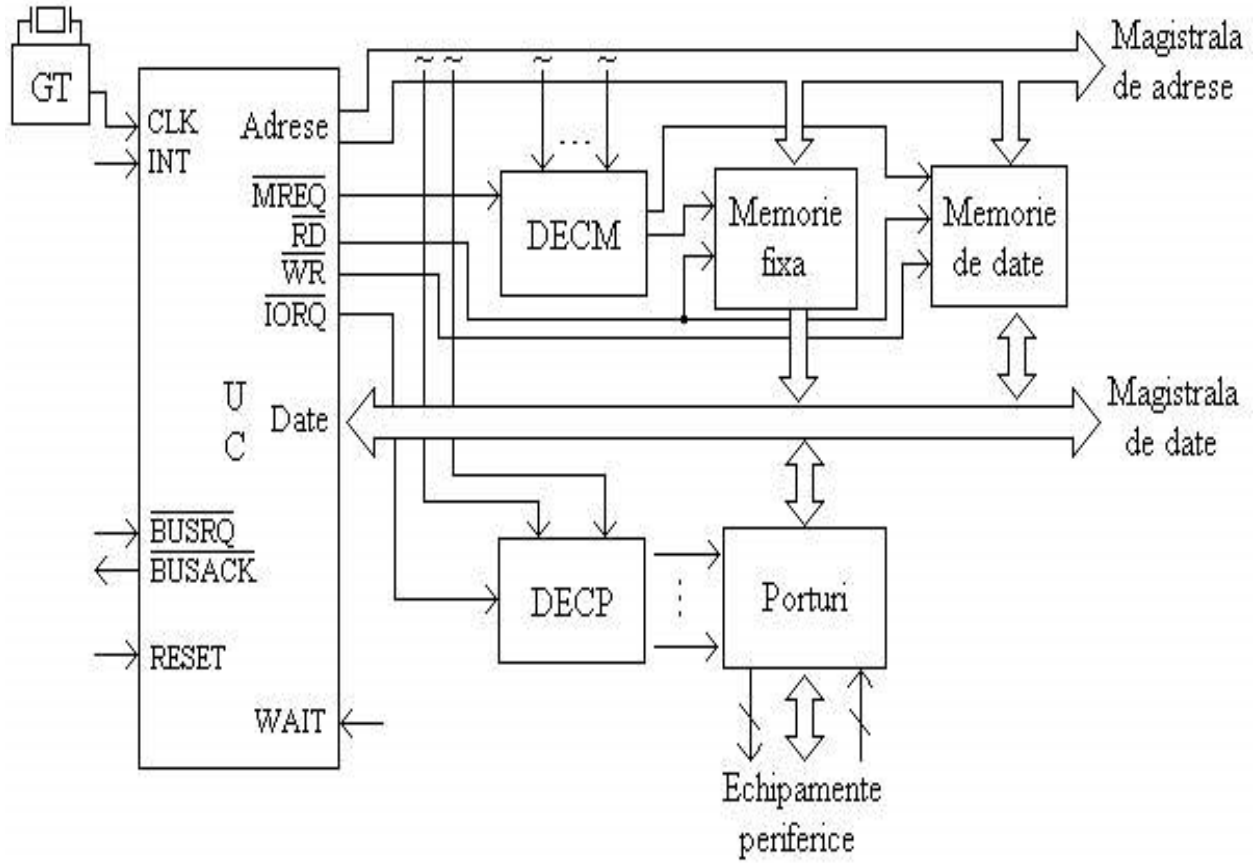
2.Uniformitate logica - nivelul active al semnalelor sa fie acelasi.

3.Uniformiate elastic - liniile sa aiba aceleasi caracteristici electrice

4.Blocuri cu iesiri cu 3 stari - este esental ca blocurile conectate la magistrala sa aiba posibilitate de a trece in de impendanta inalta atunci and nu transmit semnale , astfel incat alte blocuri sa poata folosi magistrala fara interferente.

Portile logice normale cu 2 stari nu au iesiri tristate ceea ce inseamna ca ele nu pot trece in stare de implendanta inalta, ceea ce poate provoca conflicte atunci cand mai multe componente incearca sa utilizeze aceasi linie

5. Prezentați și comentați schema bloc a unui microsistem digital.



Unitatea centrală (UC): microprocesor sau microcontroler + alte circuite (generator de tact, generator al semnalului de inițializare, amplificarea și demultiplexarea magistralelor);

- Memoria fixă:
  - implementată cu circuite ROM, OTP, EPROM, EEPROM sau Flash,
  - memorează programe de sistem sau aplicative,
  - la PC conține doar un set de rutine de bază pentru comunicarea procesorului cu periferia, programe de test, variabile care fixează anumite particularități funcționale și un program încărcător care va încărca sistemul de operare de pe suport extern în memoria de tip RAM și îl va lansa în execuție.
- Memoria de scriere/ citire (RAM):
  - pentru memorări temporare iar la PC și pentru memorarea, în timpul unei sesiuni de lucru, a sistemului de operare,
  - SRAM (RAM static) și DRAM (RAM dinamic).
- Porturi de intrare/ieșire:

- asigură interfaţa dintre unitatea centrală şi echipamentele de intrare/ieşire,
- converteşte informaţia din formatul unităţii centrale în cel al perifericelor şi invers.

- Decodificatorul de memorie (DECM):
  - generează semnale de selecţie pentru circuitele de memorie,
  - are ca intrări linii din magistrala de adrese.
- Decodificatorul de porturi (DECP):
  - generează semnale de selecţie pentru circuitele de memorie,
  - are ca intrări linii din magistrala de adrese.

Magistrale externe de adrese, date, comandă şi control.

6 Câte unități interne are microprocesorul 8086 și care este rolul lor?

- Unitatea de interfaţare la magistrală asigură legătura microprocesorului cu exteriorul; este solicitată de toate celelalte unităţi;
- Unitatea de aducere în avans a instrucţiunilor implementează o facilitate existentă şi la microprocesorul 8086: atunci când nu se execută vreun acces la magistrală, unitatea aduce în avans instrucţiuni şi le depune într-un şir de instrucţiuni, de 16 octeţi, economisindu-se astfel timpul de aducere a codului instrucţiunii;
- Unitatea de decodificare a codului instrucţiunii extrage codurile de instrucţiune din unitatea de aducere în avans a instrucţiunilor şi le decodifică; necesită o perioadă de tact pentru a decodifica un octet de cod;
- Unitatea de execuţie cuprinde unitatea aritmetică şi logică, setul de registre şi dispozitivul de comandă şi control care conduce întreaga activitate a microprocesorului;
- Unitatea de segmentare translatează adresa logică în adresă liniară şi realizează protecţia;
- Unitatea de paginare preia, dacă este activată, adresa liniară furnizată de unitatea de segmentare şi o transformă în adresă fizică; dacă nu este activată, adresa liniară devine adresă fizică; cuprinde o memorie cache pentru paginare, în scopul creşterii vitezei.

7.Descrieți calea prin care microprocesorul 80386 ajunge la o locație de memorie considerând că mecansimul de paginare nu este activ.

Dacă mecanismul de paginare nu este activ, microprocesorul 80386 ajunge la o locație de memorie folosind segmentarea. În acest caz, adresa logică este transformată direct în adresa fizică, fără a trece prin procesul de paginare.

Etapele transformării adresei logice în adresă fizică

1. Generarea adresei logice
    - O instrucțiune accesează o locație de memorie folosind o adresă logică, formată din:
        - Selectorul de segment (înregistrele CS, DS, SS, ES, FS, GS)
        - Deplasamentul (offset-ul furnizat de instrucțiune)
2. Determinarea adresei liniare
    - Selectorul de segment este folosit pentru a accesa Tabela Descriptorilor Globali (GDT) sau Tabela Descriptorilor Locali (LDT).
    - Descriptorul segmentului conține:
        - Baza segmentului (Base Address)
        - Limita segmentului
    - Adresa liniară se obține prin formula:
      Adresa˘ liniara˘=Baza segmentului+Deplasament\text{Adresă liniară} = \text{Baza segmentului} + \text{Deplasament}
    - Dacă deplasamentul depășește limita segmentului, se generează o excepție de protecție generală.
3. Conversia în adresă fizică
    - Deoarece paginarea nu este activată, adresa liniară este folosită direct ca adresă fizică și trimisă către magistrala de adrese pentru accesarea memoriei RAM.

Concluzie

Dacă mecanismul de paginare nu este activ, adresa logică este transformată în adresă fizică doar prin segmentare. Astfel, procesorul accesează memoria direct, fără a mai trece printr-un tabel de pagini.

8 Câtă memorie virtuală și câtă memorie fizică poate fi gestionată de un task la microprocesorul 80386? Justificați.

La microprocesorul 80386, fiecare task poate gestiona:

- Memorie virtuală: 64 Terabytes (To)
- Memorie fizică: 4 Gigabytes (Go)

Justificare

Microprocesorul 80386 introduce un mecanism avansat de gestionare a memoriei, care combină segmentarea și paginarea.

1. Segmentarea
    - Fiecare task poate avea până la 16.384 segmente (8.192 segmente comune + 8.192 segmente proprii).
    - Dimensiunea maximă a unui segment este de 4 GB.
    - Astfel, memoria virtuală maximă per task este: 16.384×4GB=64TB16.384 \times 4GB = 64TB16.384×4GB=64TB
2. Paginarea
    - Dacă este activată, segmentarea generează adrese liniare, care sunt apoi transformate în adrese fizice prin paginare.
    - Adresa fizică rezultată este pe 32 de biți, ceea ce permite adresarea a 2^{32} octeți, adică 4 GB de memorie fizică accesibilă per task.

Concluzie

- Memoria virtuală este determinată de segmentare și poate ajunge la 64TB.
- Memoria fizică este limitată de paginare la 4GB.
- Aceste mecanisme oferă protecție, separare între taskuri și suport pentru multitasking, fiind utilizate de sistemele de operare moderne.

9. Care este calea prin care un task nou ajunge la segmentele proprii de memorie?

Câmpul LDT(Local Descriptor Table) din Segment Stare Task(TSS) conține informații prin care procesorul ajunge la segmentele locale ale task-ului apelat. Atunci când procesorul apelează alt task, ajunge la segmentul de stare al task-ului repesctiv, se duce la câmpul LDT, extrage informația, le încadrează în

LDTR(Local Descriptor Table Registers) și prin acest mecanism ajunge la LDT-ul corespunzător noului task și, astfel, la segmentele noului task.

///
Atunci când procesorul comută către un nou task, acesta trebuie să acceseze segmentele de memorie specifice acelui task. Acest proces se realizează prin intermediul **Segmentului Stare Task (TSS)**, care conține un câmp special numit **LDT**.

În momentul schimbării task-ului, procesorul accesează TSS-ul noului task și citește câmpul LDT, care conține informații despre segmentele acestuia. Aceste informații sunt încărcate în registrul **LDTR (Local Descriptor Table Register)**, care permite procesorului să identifice și să acceseze **Tabela Descriptorilor Locali (LDT)** corespunzătoare task-ului apelat. Prin acest mecanism, procesorul poate accesa segmentele de memorie necesare noului task și continua execuția acestuia.

10 Descrieți calea prin care microprocesorul 80386 ajunge la o locație de memorie considerând că mecansimul de paginare este activ.

Când mecanismul de paginare este activ, microprocesorul **80386** accesează o locație de memorie în două etape: **segmentare** și **paginare**.

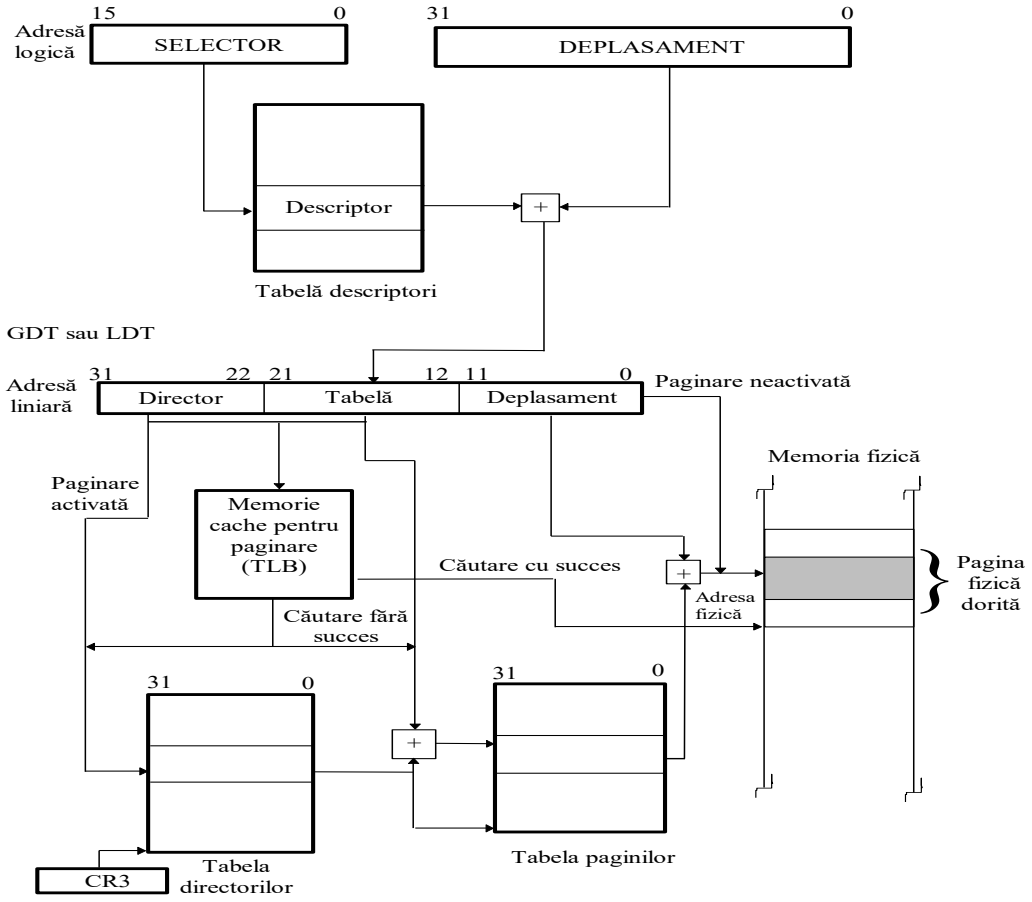
În prima etapă, segmentarea transformă **adresa logică** într-o **adresă liniară** folosind tabela descriptorilor (GDT sau LDT). Adresa liniară se calculează adunând baza segmentului cu deplasamentul.

În a doua etapă, paginarea transformă **adresa liniară** într-o **adresă fizică** folosind două tabele: **Tabela Directorilor de Pagini (PDT)** și **Tabelul Paginilor (PT)**. Adresa liniară este împărțită astfel: primii 10 biți selectează o intrare în PDT, următorii 10 biți selectează o pagină în PT, iar ultimii 12 biți reprezintă deplasamentul în pagină. Adresa fizică finală este obținută prin combinarea bazei paginii fizice cu acest deplasament.

Acest mecanism permite utilizarea eficientă a memoriei și izolarea proceselor, dar introduce o ușoară întârziere din cauza accesului suplimentar la memorie.

11. Prezentați schema bloc a adresării memoriei la microprocesorul 80386.

În partea de sus a circuitului avem prezentat schematic mecanismul de segmentare, iar în partea de jos – mecanismul de paginare cu memoria cache.



12. De ce este necesară amplificarea liniilor unui microprocesor? Prezentați soliții pentru linii uni și bidirecționale.

Amplificarea liniilor unui microprocesor este necesară pentru a asigura transmiterea corectă și fiabilă a semnalelor între procesor și celelalte componente ale sistemului. Aceasta este importantă deoarece microprocesorul trebuie să conducă mai multe dispozitive, precum memoria și perifericele, ceea ce poate duce la suprasolicitarea liniilor de semnal. De asemenea, semnalele slăbesc pe măsură ce distanța crește, iar amplificarea contribuie la menținerea clarității acestora. În plus, protecția microprocesorului împotriva suprasarcinilor și asigurarea unei sincronizări corecte în sistem sunt alte motive pentru care amplificarea este necesară.

Pentru liniile unidirecționale, cum sunt cele de adresă și de control, soluția optimă este utilizarea bufferelor tristat, precum circuitul 74LS244. Acestea permit conectarea mai multor dispozitive la magistrale fără a suprasolicita microprocesorul. De asemenea, prin utilizarea unui control de activare, bufferul poate comuta între starea activă și starea de impedanță ridicată (Hi-Z), permițând astfel utilizarea eficientă a resurselor sistemului.

În cazul liniilor bidirecționale, utilizate în special pe magistralele de date, soluția eficientă este utilizarea bufferelor bidirecționale cu control de direcție, cum ar fi 74LS245. Aceste circuite permit transmiterea datelor în ambele sensuri, iar direcția de transfer este controlată de un semnal special (DIR). Astfel, microprocesorul poate comunica eficient cu memoria și perifericele, evitând conflictele pe magistrală.

Prin utilizarea acestor metode, se obține o îmbunătățire a calității semnalelor, protecția microprocesorului și posibilitatea conectării unui număr mai mare de dispozitive fără a compromite performanța sistemului.

13. Ce înseamnă multiplexarea liniilor unui microprocesor? Este necesară demultiplexarea? Dacă da, de ce și cum?

Multiplexarea liniilor unui microprocesor reprezintă utilizarea aceloraşi linii fizice pentru transmiterea mai multor tipuri de informații în momente diferite. Aceasta este o tehnică utilizată pentru a reduce numărul de pini ai microprocesorului și, implicit, complexitatea circuitului integrat. De exemplu, în cazul microprocesorului 8086, liniile AD0 – AD15 sunt multiplexate, ceea ce înseamnă că ele sunt folosite atât pentru transmiterea adreselor, cât și pentru transferul datelor, dar în momente diferite ale ciclului de execuție.

Demultiplexarea este necesară deoarece, la un moment dat, memoriile și perifericele trebuie să primească separat atât adresa locației de memorie accesată, cât și datele care trebuie citite sau scrise. Atunci când microprocesorul transmite o adresă, aceasta trebuie menținută stabilă pe durata întregului ciclu de acces la memorie. Întrucât aceleași linii sunt utilizate ulterior pentru transmiterea datelor, este nevoie de un mecanism care să „memoreze” adresa transmisă înainte de schimbarea rolului liniilor.

Pentru demultiplexare, se utilizează registre de tip latch, cum ar fi 74LS373 sau 74LS374. Aceste circuite rețin informația de adresă în momentul în care aceasta

este disponibilă pe magistrală, permițând astfel utilizarea ulterioară a liniilor pentru date, fără a pierde informația inițială.

Astfel, multiplexarea permite reducerea numărului de pini ai microprocesorului, iar demultiplexarea este necesară pentru a separa informațiile și a asigura funcționarea corectă a sistemului.

14.Ce înseamnă existența a mai multor moduri de lucru la un microprocesor? Exemplificați la un microprocesor.

Existența mai multor moduri de lucru la un microprocesor înseamnă că acesta poate funcționa în diferite configurații, fiecare fiind adaptată pentru anumite tipuri de aplicații și necesități. Această flexibilitate permite microprocesorului să ofere compatibilitate cu software mai vechi, să îmbunătățească gestionarea memoriei și să asigure protecție între procese.

Un exemplu de microprocesor care utilizează mai multe moduri de lucru este Intel 80386. Acesta dispune de trei moduri principale: modul real, modul protejat și modul virtual 8086.

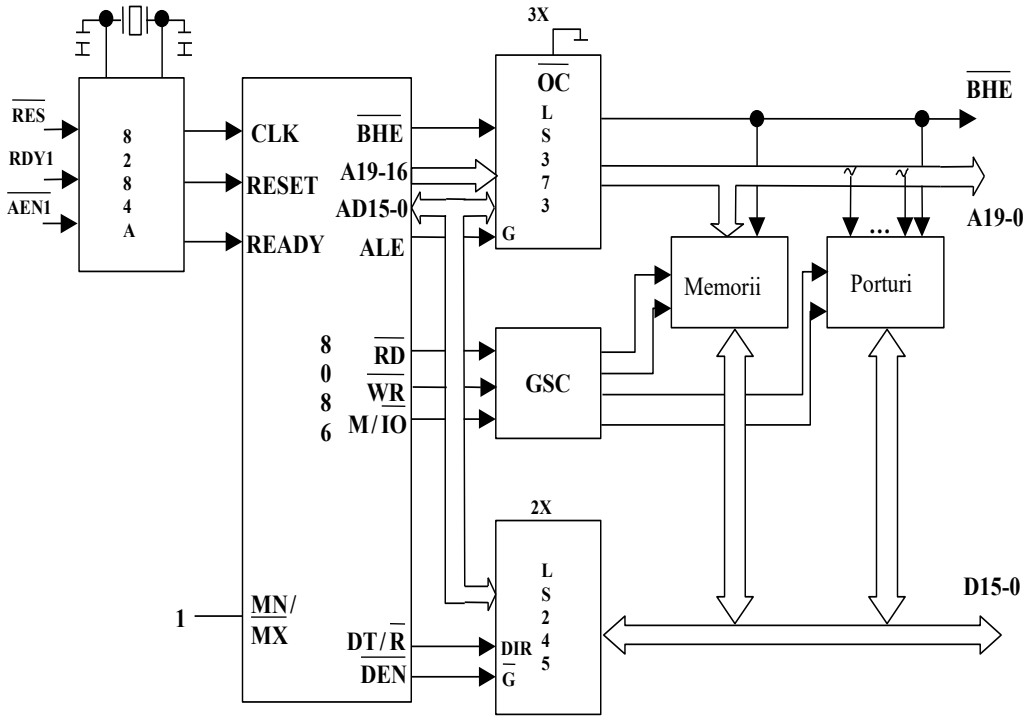
În modul real, microprocesorul se comportă ca un 8086, fiind capabil să adreseze doar 1 MB de memorie și utilizând un sistem simplu de segmentare, fără mecanisme de protecție între procese. Acest mod este folosit în principal pentru a rula software mai vechi, precum sistemul de operare MS-DOS.

În modul protejat, microprocesorul își valorifică pe deplin capacitățile, permițând accesul la 4 GB de memorie fizică și 64 TB de memorie virtuală. Acest mod introduce protecție avansată între procese, utilizând mecanisme precum segmentarea avansată și paginarea. De asemenea, permite rularea mai multor sarcini simultan prin suportul pentru multitasking hardware. Acesta este modul utilizat de sistemele de operare moderne, precum Windows și Linux.

În modul virtual 8086, microprocesorul poate rula simultan mai multe aplicații destinate arhitecturii 8086, însă în interiorul unui sistem de operare modern. Acest mod este o combinație între compatibilitatea oferită de modul real și protecția oferită de modul protejat.

Prin existența acestor moduri, microprocesorul 80386 poate oferi atât compatibilitate cu software mai vechi, cât și performanță ridicată și securitate pentru aplicațiile moderne. Această caracteristică a devenit esențială în dezvoltarea arhitecturilor de calcul, permițând tranziția treptată între generațiile de software și hardware.

15. Prezentați schema unității centrale cu microprocesorul 8086 în modul minim.



16. Dați exemple pentru un sistem multiprocesor cu resurse partajate.
Un sistem multiprocesor cu resurse partajate este un sistem în care mai multe procesoare lucrează împreună și împart anumite resurse, cum ar fi memoria, magistralele de date și perifericele. Acest tip de arhitectură este utilizat pentru a îmbunătăți performanța și fiabilitatea sistemului.

Un exemplu de astfel de sistem este un **server multiprocesor**, unde două sau mai multe unități centrale de procesare (CPU) împart memoria RAM și dispozitivele de stocare. De exemplu, **serverele bazate pe procesoare Intel Xeon** sau **AMD EPYC** sunt proiectate să funcționeze în configurații cu mai multe procesoare fizice, fiecare având acces la aceeași memorie și la aceleași resurse de intrare/ieșire.

Un alt exemplu este arhitectura **NUMA (Non-Uniform Memory Access)**, utilizată în supercomputere și servere de înaltă performanță. Într-un astfel de sistem, fiecare procesor are propria sa memorie locală, dar poate accesa și memoria altor procesoare, ceea ce permite o gestionare eficientă a resurselor.

De asemenea, sistemele integrate multiprocesor, cum ar fi **microcontrolerele utilizate în industria auto** sau în sistemele de control robotic, pot avea mai multe unități de procesare care partajează resurse pentru a îndeplini sarcini în timp real, cum ar fi procesarea senzorilor și controlul motoarelor.

Astfel, sistemele multiprocesor cu resurse partajate sunt utilizate într-o gamă largă de aplicații, de la servere și supercomputere, până la sisteme industriale și dispozitive integrate.

17. Care este principalul dezavantaj al unui sistem multiprocesor cu resurse partajate și ce soluție există pentru gestionarea acestuia?
Principalul dezavantaj al unui sistem multiprocesor cu resurse partajate este competiția pentru accesul la magistrala comună, ceea ce poate duce la blocaje și scăderea performanței sistemului.

Pentru a gestiona acest dezavantaj, se utilizează **arbitrarea accesului la magistrală**, realizată de circuitul **8289**, care coordonează cererile procesoarelor și evită conflictele. Acesta poate funcționa în trei moduri: **arbitraj paralel, serial și rotativ**. O altă soluție este utilizarea tehnicii **NUMA (Non-Uniform Memory Access)**, unde fiecare procesor are o memorie locală pentru a reduce congestionarea magistralei comune.

18. Este necesară existența unui decodificator de memorii într-un microsistem digital? Justificați răspunsul.

Un **decodificator de memorii** este necesar într-un microsistem digital pentru a gestiona corect accesul la memorie. Acesta permite **selecția precisă a modulelor de memorie**, evitând conflictele pe magistrală și optimizând utilizarea spațiului de adresare. De asemenea, **simplifică circuitul**, reduce numărul de semnale de control și îmbunătățește performanța sistemului. Fără un decodificator, microprocesorul nu ar putea accesa eficient memoria, ceea ce ar duce la erori și funcționare defectuoasă.

19. Descrieți decodificarea completă. Avantaje și dezavantaje.

- se construieşte un tabel având ca şi coloane rangurile de adrese şi ca linii configuraţii care arată adresa de început şi de sfârşit a fiecărei zone ocupate de câte un circuit;
- pentru fiecare circuit, se inspectează coloanele, pornind de la rangurile mai semnificative spre cele mai puţin semnificative şi se stabilesc acele ranguri ce rămân nemodificate oricare ar fi locaţia adresată în respectiva zonă;
- o funcţie combinaţională a acestor intrări va individualiza zona respectivă şi va constitui ecuaţia pentru semnalul de selecţie a circuitului care o acoperă.

Avantaje:

- Fiecărei locații adresabile din memorie îi corespunde o adresă unică.

Dezavantaje:

- Toate liniile de adresă trebuie folosite în procesul decodificării.

20. Descrieți decodificarea incompletă. Avantaje și dezavantaje.

- se construieşte tabelul ca în cazul anterior dar funcţia sau funcţiile care vor genera semnalele de selecţie nu vor utiliza toate liniile de adresă ce rămân nemodificate oricare ar fi locaţia adresată din zona respectivă;
- aceasta va duce la o simplificare a decodificatorului dar un acelaşi circuit, sau diviziune, va ocupa mai multe zone de memorie de aceeaşi capacitate ca cea a circuitului;
- dacă nu se consideră o linie de adresă un acelaşi circuit va ocupa 2 zone de memorie, dacă nu se consideră 2 linii de adresă un acelaşi circuit va ocupa 4 zone de memorie ş.a.m.d.

Avantaje:

- Simplă și ieftină;
- Unele linii de adresă nu sunt folosite în procesul decodificării.

Dezavantaje:

Un circuit proiectat ocupă mai multe zone de memorie

21.

22. Se pot conecta porturi la un procesor care nu are sistem de intrare/ieșire? Dacă nu, de ce? Dacă da, cum?

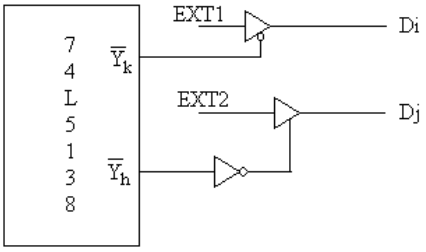
Da, în spaţiul de memorie al microprocesorului: comunicarea microprocesorului cu

ele se va face cu instrucţiuni de lucru cu memoria întrucât microprocesorul nu va face distincţie între porturi şi memorie, în sistem va exista doar decodificatorul de memorie iar porturilor le sunt alocate adrese sau zone din spaţiul de memorie.

23. Prezentați și comentați soluția pentru portul realizat cu porți. Pentru ce tip de port se recomandă? De ce?

Este folosită pentru porturile de intrare;

- Ieşirile porţilor se conectează la linii de date ale microprocesorului, ca urmare este necesar să aibă facilitatea de a 3 – a stare;
- Semnalele care se citesc se conectează la intrările de date ale porţilor;
- Intrările de validare ale porţilor sunt comandate de una sau mai multe ieşiri dintr - un decodificator de porturi;
- Semnalele care se citesc sunt generate din exterior şi se presupune că circuitul care le generează le şi memorează până când este necesar; în caz contrar sunt prevăzute bistabile;
- Citirea se face în timpul ciclului de intrare.



Este folosit pentru porturile de intrare

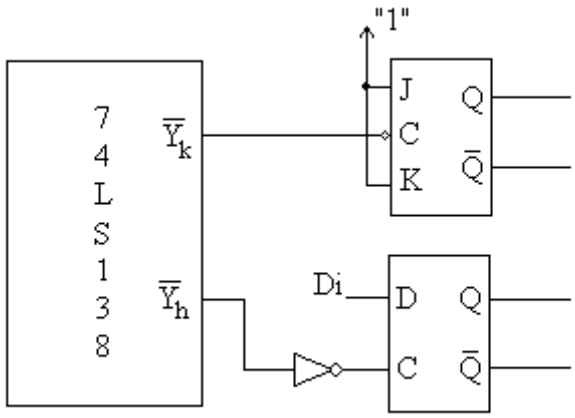
24. Prezentați și comentați soluția pentru portul realizat cu bistabile. Pentru ce tip de port se recomandă? De ce?

Portul implementat cu bistabile

- Este folosită pentru porturile de ieşire;
- Există mai multe variante:
  - cu bistabile:
    - de tip JK;
    - de tip D;
  - cu bistabile care comută:

- pe nivelul impulsului de tact;
- pe frontul impulsului de tact:
  - anterior;

posterior



Este folosit pentru porturile de ieșire, întrucât la ieșire avem nevoie de semnale de durată ca să vedem un anumit rezultat. Prin urmare, această comandă de ieșire trebuie memorată, facilitate oferită de bistabile.

25. Ce înseamnă comanda unui semnal prin program?

a. Citirea unui semnal din exteriorul microsistemului: soluţia este cea prezentată la citirea unui semnal prin intermediul unui port implementat cu o poartă:
   i. Semnalul este legat la intrarea de date a unei porţi cu 3 stări;
   ii. Intrarea de validare a porţii este comandată de o ieşire a unui decodificator de porturi;
   iii. Ieşirea porţii este legată la o linie a magistralei de date a unităţii centrale;
   iv. Citirea se face cu o instrucţiune IN şi apoi se face ramificarea.

b. Poziţionarea unui semnal la 1 sau 0 logic: soluţia este cea prezentată la implementarea unui port cu un bistabil:
   i. Valoarea semnalului este transmisă la intrarea de date a unui bistabil prin intermediul unei linii a magistralei de date;
   ii. Tactul bistabilului este comandat de o ieşire a unui decodificator de porturi;
   iii. Iesirea bistabilului este semnalul care trebuie comandat;
   iv. Poziţionarea semnalului se face cu o instrucţiune OUT.

26.

27. Prezentați și comentați caracteristicile de bază ale interfeței RS232.

Nivelele de tensiune:

- Nu sunt TTL ci EIA şi au următoarele valori:
  - - 25V ÷ - 3V pentru “1” logic şi
  - + 3V ÷ + 25V pentru “0” logic;
- Sunt necesare circuite de conversie TTL → EIA şi EIA → TTL: 232;

- Direcţia de deplasare a datelor:
  - Simplex (unidirecţionale), semi-duplex sau half duplex (bidirecţionale dar la momente de timp diferite) şi duplex sau full duplex (bidirecţionale ce pot avea loc simultan);
- Viteza:
  - Mică;
  - Există două unităţi de măsură a vitezei transferului: bit pe secundă (bps) şi baud; dacă transferul este digital atunci 1 bps = 1 baud;
  - Treptele de viteză mai utilizate sunt: 300, 600, 1200, 2400, 4800, 9600, 19200, 38400, 57600, 115200 mai rar 230400, 460800 bps.
- Transferul serial de tip RS232 este util atunci când există distanţe mari (peste 3 m) între echipamentele care comunică.
- Există două motive care susţin această recomandare: costul şi rezistenţa la perturbaţii:
  - Costul este determinat de numărul firelor din cablul care leagă cele 2 echipamente: dacă acest număr este mai mic, costul va fi mai redus;
  - Transferul serial de tip RS232 are o rezistenţă mai mare la perturbaţii decât cel paralel din două motive:
    - posibilitatea de perturbare a liniilor scade dacă numărul acestora este mai mic şi
    - distanţa dintre nivelele de tensiune corespunzătoare celor 2 nivele logice este mai mare decât la transferul paralel.
- Sincronizarea între emiţător şi receptor:
  - Transfer sincron;
  - Transfer asincron: transfer pe caracter, un caracter poate fi transmis în orice moment;
  - Forma de undă a unui caracter:
  - Caracteristici:
    - Sincronizare la nivel de caracter prin bitul START;
    - 5 – 8 biţi de date;
    - Opţional bit de paritate;
    - 1, 1.5, 2 biţi de STOP.
  - Rejecţia impulsurilor de START false şi citirea informaţiei la receptor:
  - Avantaje:

        - Transmiţătorul poate transmite un caracter oricând doreşte;
        - Simplu de implementat.
    - Dezavantaje:
        - Se transferă şi informaţie inutilă (de cadrare): minim 20%;
        - Rată de transfer mică.
    - Transfer sincron: transfer pe bloc, transferul se face la nivel de bloc sau de mesaj.
    - Avantaj:
        - Asigură viteză mai mare datorită:
            - Existenţei tactului;
            - Elimină informaţia de cadrare la nivel de caracter.
    - Dezavantaje:
        - Sunt necesare caractere de sincronizare deci dacă blocul este scurt, sub 8 caractere, modul sincron nu mai este eficient;
        - Linia suplimentară de tact.
- Codificarea informaţiei alfanumerice: prin coduri:
    - BAUDOT: folosea doar 5 biţi pentru codificarea unui caracter, ceea ce a limitat la 32 numărul de caractere ce puteau fi transferate;
    - EBCDIC;
    - ASCII ("American Standard Code for Information Interchange"): 96 de octeţi corespund la litere, numere sau caractere speciale, iar 32 de octeţi corespund la caractere de comandă.

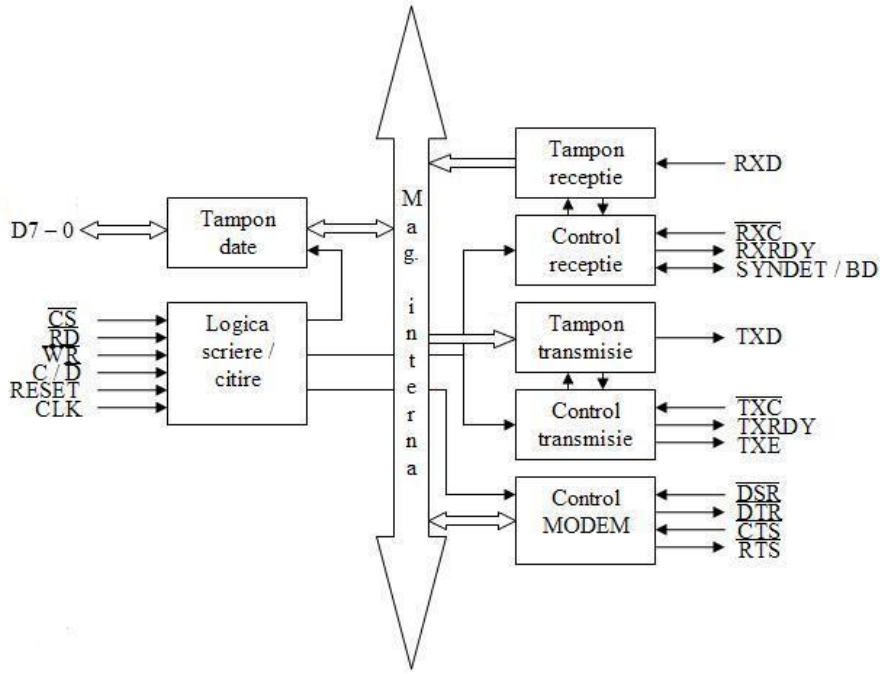
28. Ce este circuitul 8251? Prezentați caracteristici ale sale.
Circuitul specializat programabil 8251

- Specializat pentru transferurile seriale;
- Face parte din categoria circuitelor de tip USART ("Universal Synchronous Asynchronous Receiver Transmitter"); moduri de lucru:
    - Sincron;
    - Asincron;
- Poate să primească un octet în paralel de la unitatea centrală, să – l serializeze şi să – l transmită la un echipament serial;
- Poate să preia de pe linie, de la un echipament periferic serial, un octet, să – l asambleze şi să – l predea, în paralel, unităţii centrale;
- Circuitul comunică unităţii centrale când are un caracter gata pentru ea sau când a terminat de transmis un octet şi poate prelua altul; poate comunica:

- Prin program;
- Prin întreruperi;

Transmisie şi recepţie cu dublu tampon.

29 Prezentați și comentați structura internă a circuitului 8251.

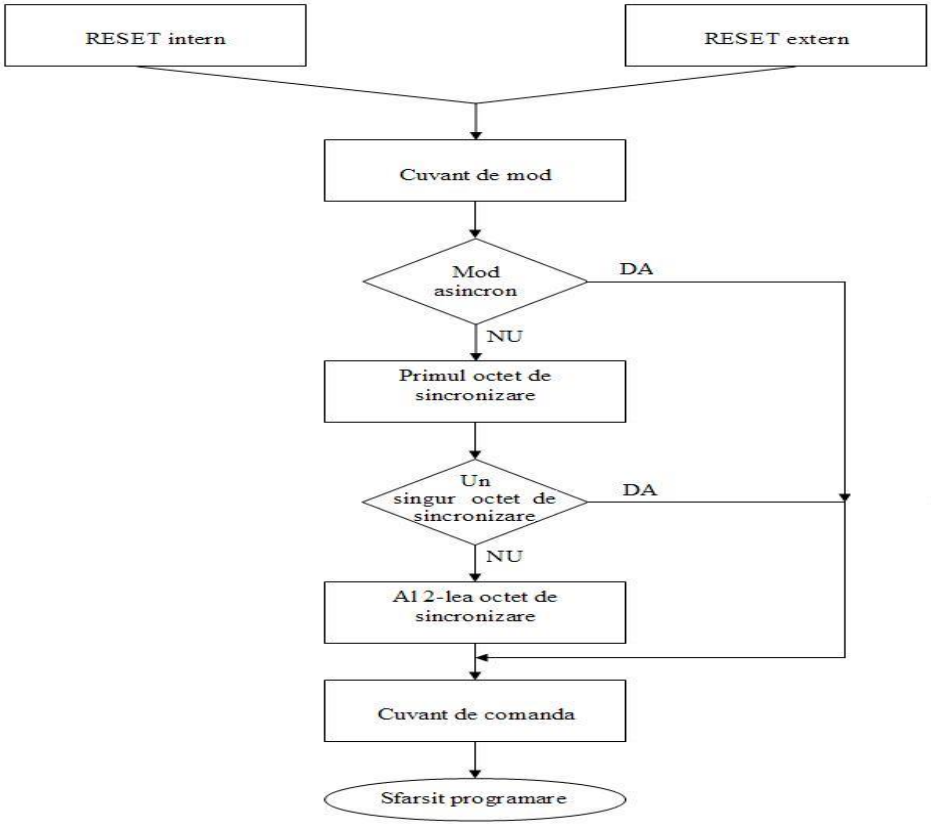


Există o magistrală internă la care se leagă, pe de o parte, blocuri pentru legătura cu procesorul, tamponul de date, logica de scriere/citire(toate semnalele de comandă intră aici), iar pe de altă parte – blocuri pentru comunicare cu linia serial. Pentru recepție există un tampon și o parte de control.

a. C//D este intrarea prin care se alege unul din cele 2 porturi interne ale circuitului: portul de comandă/stări, ptr. C = 1 și portul de date ptr. C = 0; aici se leagă o linie din magistrala de adrese (de obicei A1 sau A2);
b. RXD – linia pe care vine informația serial;
c. ~RXC – tactul pentru recepție;
d. RXRDY – linia prin care partea de recepție anunță procesorul că are un caracter pentru el;
e. TXD – linia serial pe care pleacă datele;
f. ~TXC – tactul pentru transmisie;

TXRDY - Ieșirea prin care circuitul îi spune procesorului că are nevoie de un caracter

30 Cum se programează circuitul 8251 (nu este necesară structura cuvintelor ci doar ce anume i se comunică circuitului)?



Programarea începe cu inițializarea circuitului, care poate fi internă sau externă. Circuitul așteaptă un cuvânt de mod prin care i se spune să lucreze în mod sincron sau asincron. Dacă i se cere să lucreze în mod asincron, acesta merge pe ramura corespunzătoare, așteptând un cuvânt de comandă, după care-și încheie execuția. Dacă i se cere să lucreze în mod sincron, primește primul octet de sincronizare prin care află dacă va mai fi vreun octet de sincronizare. Dacă da, îl așteaptă și pe al doilea, după care trece la așteptarea cuvântului de comandă și își încheie ciclul de programare. Dacă nu va mai fi un alt octet de sincronizare, sare peste etapa de așteptare a acestuia, următoarele etape fiind identice cu cele descrise anterior.